DEF CON // IoT Village

Anyone Can Hack IoT

(Even Easier Now)



A Beginner's Guide to
AI-Augmented IoT Hacking

Andrew Bellini
@d1gitalandrew · Sr. Product Security Engineer

Anyone Can Hack IoT

The original talk still holds up

> **"The S in IoT still stands for Security."**

- **~15B+ IoT devices today and still heading toward ~30B**
- Cheap consumer gear is still soft: open debug ports, unencrypted firmware, root shells
- Beginners still ask the same question: "isn't this too expensive / too complicated?"
- **Answer is still NO! The on-ramp just got shorter, and the vulns are easier to find!**

Anyone Can Hack IoT

whoami

- **Andrew Bellini — @digitalandrew**
- Background in electrical engineering; designed embedded / industrial IoT before security
- Senior Product Security Engineer (full-time IoT hacking now)
- Shipped CVEs in real consumer devices
- Built a beginner IoT hacking course + the PIPA (Practical Junior IoT Tester) certification
- Taught thousands of students to find their first bug
- **Love Hacking IoT and teaching others how to**



Anyone Can Hack IoT

What changed: AI showed up at the bench

THEN (DC32)

- Mostly everything by hand
- Read every boot log
- grep firmware manually
- Stare at Ghidra alone
- Trace each call yourself

NOW

- AI agent does the grunt work
- Summarizes logs instantly
- Searches the whole FS for you
- Explains decompiled code
- Traces dataflow to the bug
- And hooked into hardware

Same methodology. The boring 80% is now automated, so you spend your time on the interesting 20%.Or work wayyyy faster.



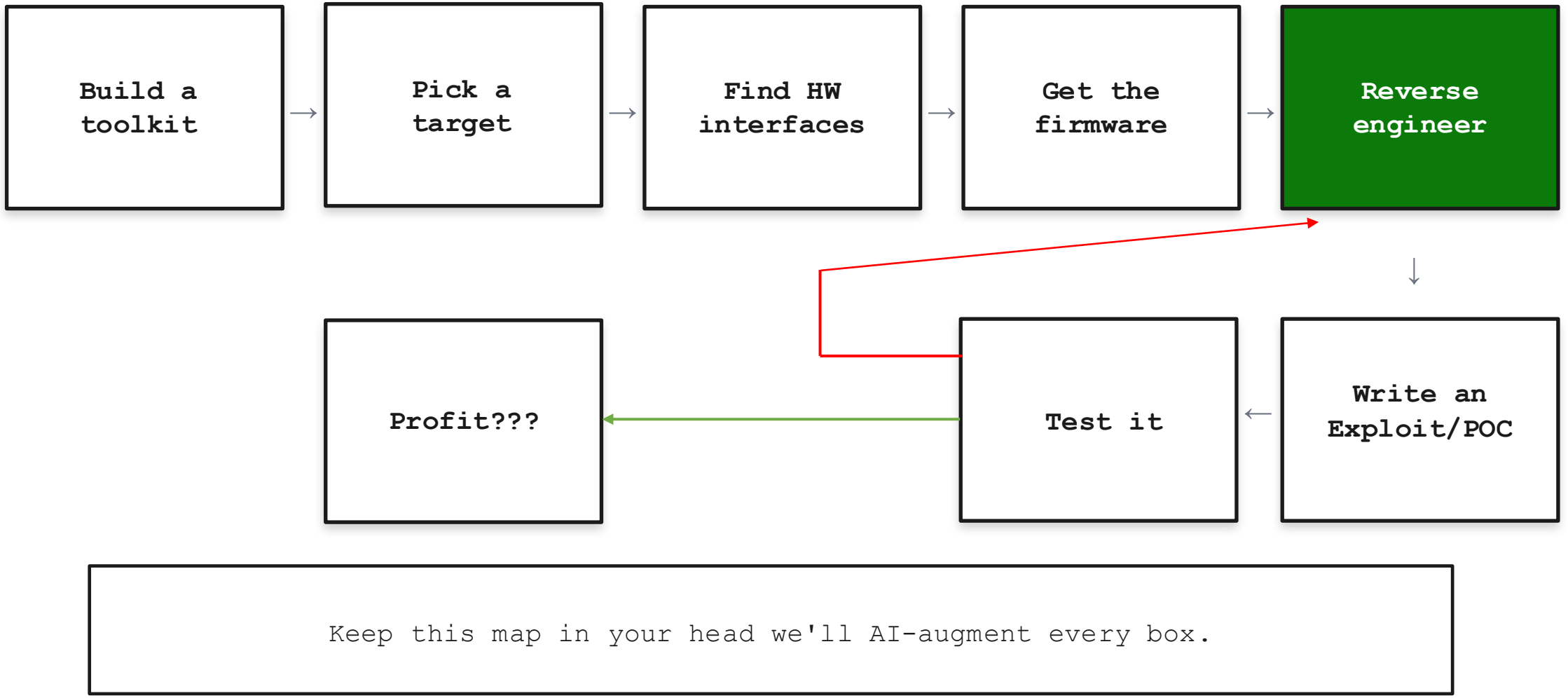
PART 1

The 5-Minute Recap

The manual methodology is still the foundation everything else builds on, we'll add AI to it.

Anyone Can Hack IoT

The methodology in one picture



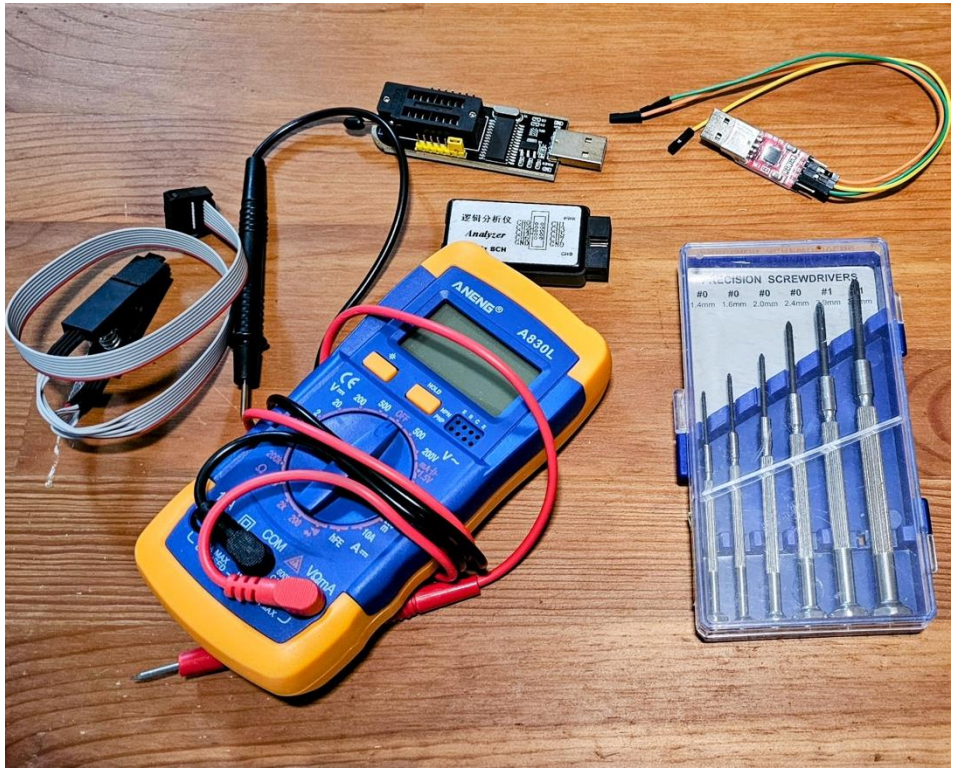
Anyone Can Hack IoT

Recap: the $20 hardware hacking toolkit

Multimeter
Find & confirm UART by watching voltage on boot **~$10**

USB-UART adapter
Talk to the device's serial console **~$2**

Flash programmer
Dump SPI flash to get firmware **~$10**



Anyone Can Hack IoT

The on-ramp: cheap + embedded Linux

- **Pick cheap consumer gear like routers & smart cameras**
- cheap = less security = open ports, no firmware encryption, no secure boot, lots of vulns to be found
- Cameras and routers almost always run embedded Linux, far friendlier than bare-metal MCUs
- Dumpster-dive: friends/family give you their old devices
- AI Augment = research based on your needs (cheap, available, past CVEs)
- **Golden rule: only test devices you're authorized to; disclose responsibly**



Anyone Can Hack IoT

Recap: UART = logs + (often) a root shell

- **TX / RX / GND — ignore VCC**
- Spot it: pins near the SoC; voltage flutters on boot, picture for Claude
- Cross TX↔RX, 115200 baud, open with screen
- Boot logs leak versions, partitions, keys, endpoints
- **Cheap devices: hit Enter → root shell, no password**
- Even with no shell, logging streams out for free
- AI augment = coding agents can drive UART
- Come to my free workshop in the IoT Village

```
$ screen /dev/ttyUSB0 115200

U-Boot 1.1.3 (Mar 2 2021)
Booting Linux...
[ 1.84] drop_bear: loading RSA host key
[ 2.11] mounting squashfs root (ro)
[ 2.63] starting /etc/init.d/rcS
BusyBox v1.19  built-in shell (ash)

# id
uid=0(root) gid=0(root)
#
```

Anyone Can Hack IoT

Recap: easy ways to get the firmware

1. **Download it**
   Vendor posts it on their support page or open S3 bucket no hardware, totally free

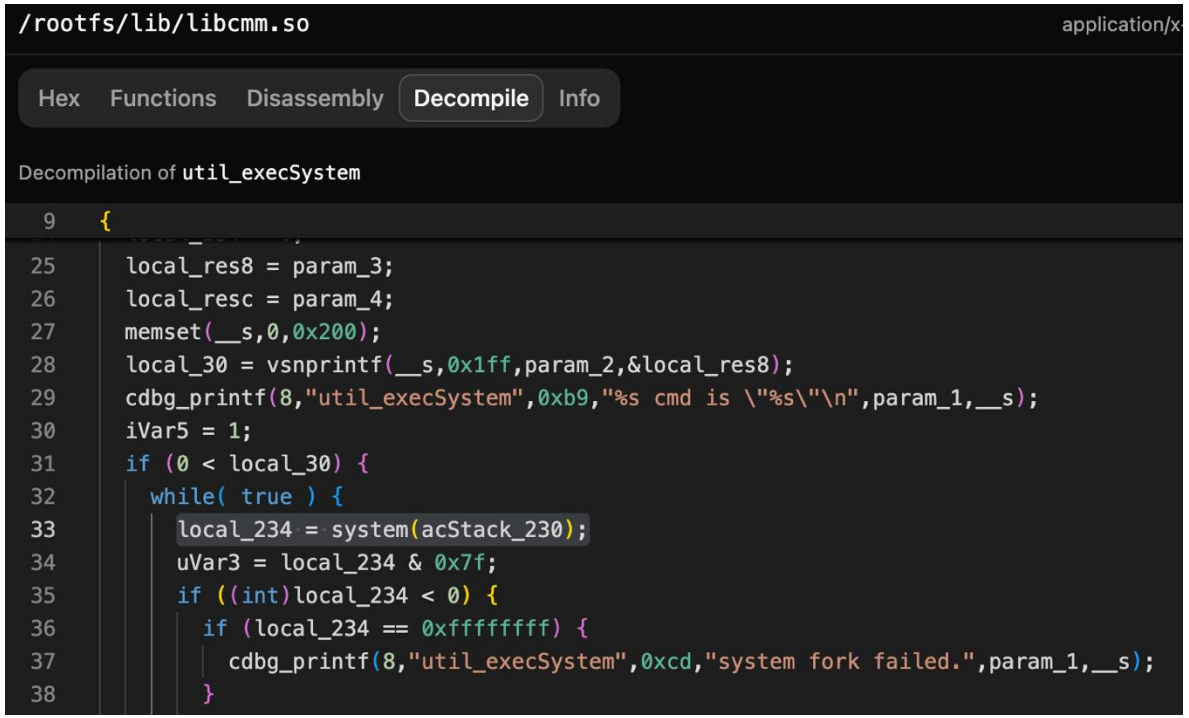
2. **Dump the SPI flash**
   8-pin chip, in-circuit read with flashrom + a cheap clip, dump over uart

Easiest path wins: if the vendor hosts it, you can do firmware RE for $0 and never touch hardware. AI can do the download research for you.

Anyone Can Hack IoT

Recap: reverse engineering → the bug

- binwalk -Me to unpack → focus on the root filesystem
- strings | grep, read scripts & configs (low-hanging fruit)
- Interact with the device, follow the logging to the binary
- Decompile the custom libs and bins in Ghidra; let print statements guide you
- AI to explain or clean up decompiled C



Command injection
user input → system() with no sanitization

Buffer overflow
user input → strcpy / memcpy / sprintf



PART 2

Where Does AI Actually Fit?

Cutting through the hype what's worth handing to an agent, and what isn't.



Anyone Can Hack IoT

My framing: force multiplier

AI is the best and worst embedded reverse engineer.

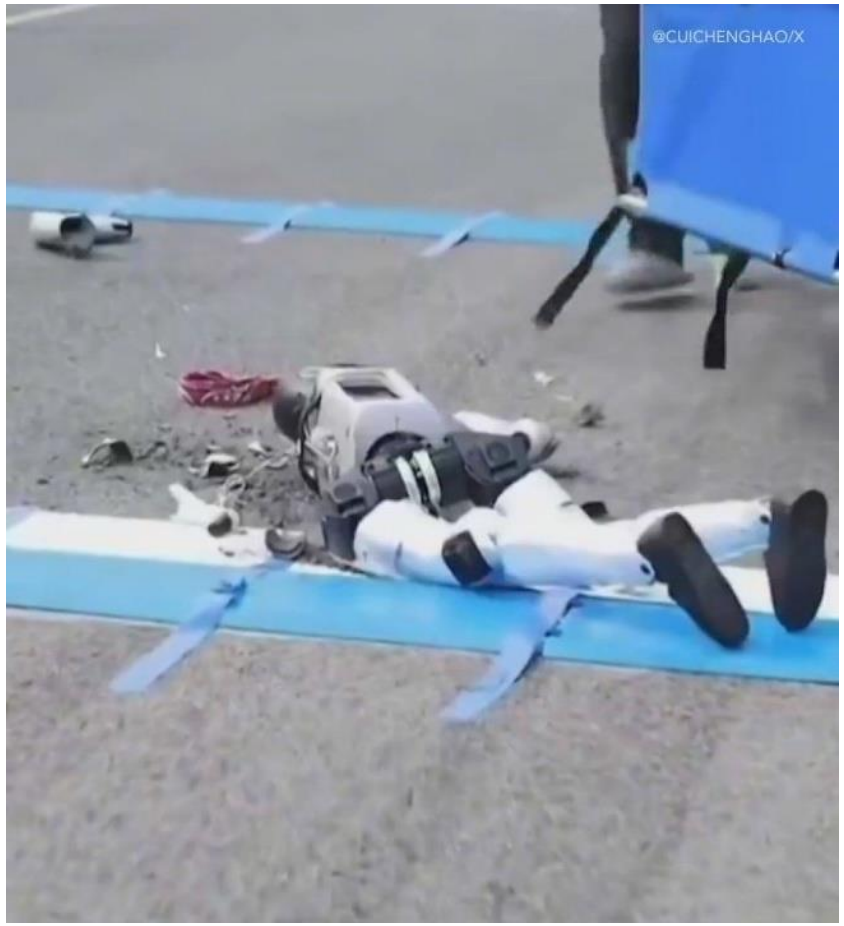
- Embedded RE is mostly reading: logs, scripts, decompiled C, docs
- That reading is the perfect job for an LLM with the right tools
- **You stay the analyst, you make the calls, AI does the legwork**
- Best results when the AI can act -> read files, run RE tools, drive hardware
- My goal is to give AI access to all the same tools I would want to use
- AI still lacks context especially in embedded about what is actually a crit/high vuln

Anyone Can Hack IoT

Still do it yourself

STILL BETTER BY HAND/HUMAN

- Soldering, connecting, opposable thumbs
- Physical probing & scope work
- **Where to prioritize spending time**
- **Judgment: What is the impact and risk**
- Embedded/IoT context
- Novel exploitation & chaining
- **Trusting the final finding**





PART 3

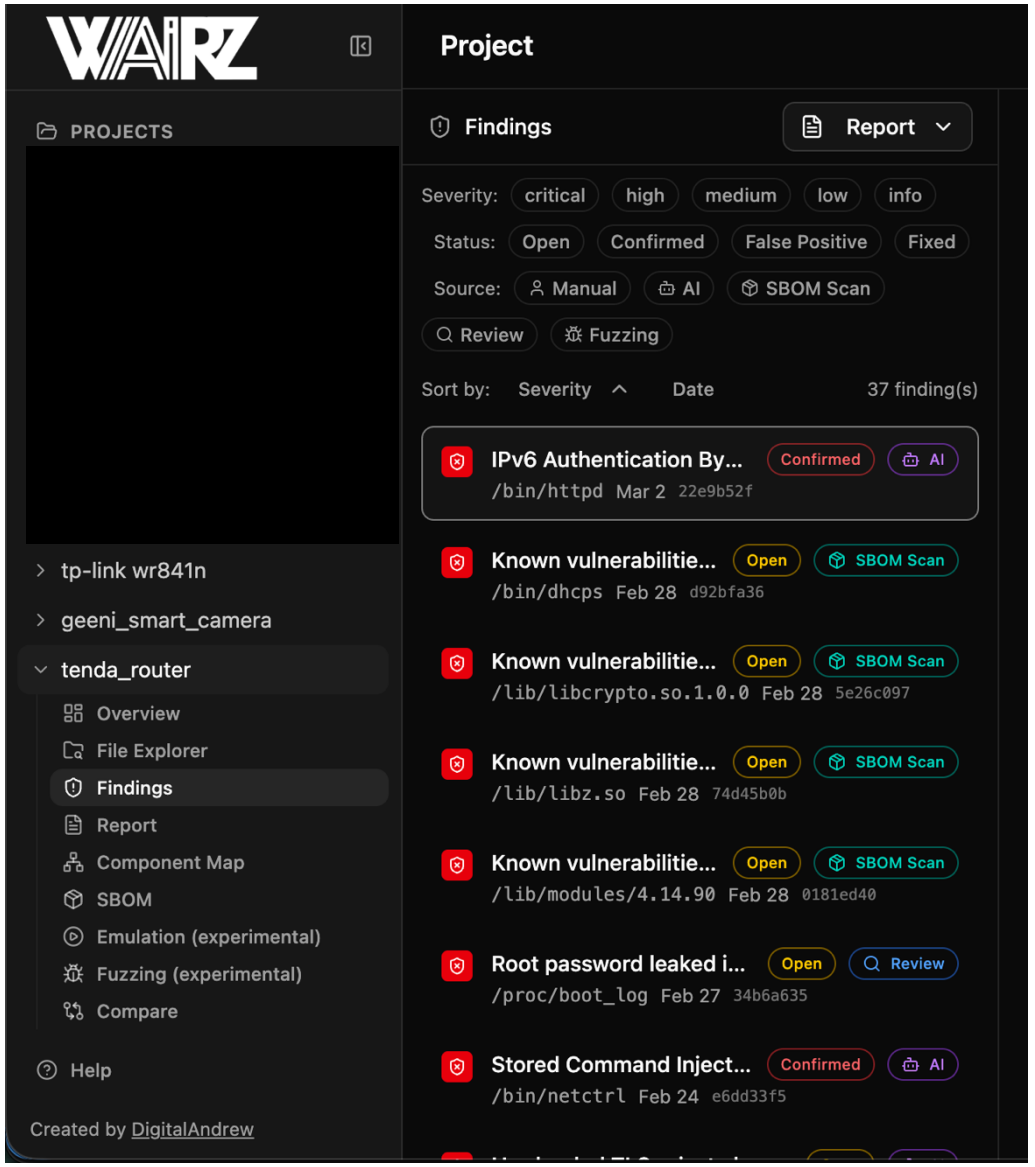
Firmware and Binary RE

AI is way better at this than me…

Anyone Can Hack IoT

Meet wairz

- **Upload firmware → unpack → explore → analyze → assess**
- Runs in your browser; backend is sandboxed in containers
- **60+ analysis tools exposed to any AI agent over MCP**
- Works with Claude, Cursor, Codex, Copilot, Gemini CLI...
- Ghidra + radare2 + QEMU + AFL++ under the hood
- **You can also use bare tool calling and RE MCPs (Ghidra, Binary Ninja, IDA)**



Anyone Can Hack IoT

wairz: unpack & orient

- Auto-extract: SquashFS, JFFS2, UBIFS, CramFS, ext
- Multi-partition; classifies Linux / RTOS / unknown
- Component map: binaries, libs & scripts linked
- **Ask the agent: "what is this device and what's custom?"**

```
❯ find the most interesting functions in httpd that are likely to contain high impact vulns

● I'll analyze the httpd binary. Let me first locate it and start binary analysis.

  Called wairz 2 times (ctrl+o to expand)

● Not stripped — good. Let me kick off analysis and list the largest functions.

  Called wairz 2 times (ctrl+o to expand)

● Classic Tenda httpd with the telltale form*/from* handlers. Let me look at the dangerous-sink imports and
  the auth handler to prioritize. Let me load the needed tools.

  Called wairz (ctrl+o to expand)

● Key sinks present: doSystemCmd, system, popen, execve, strcpy, strcat, sprintf, sscanf. Let me map their
  call sites — command-injection sinks first.

  Called wairz 4 times (ctrl+o to expand)

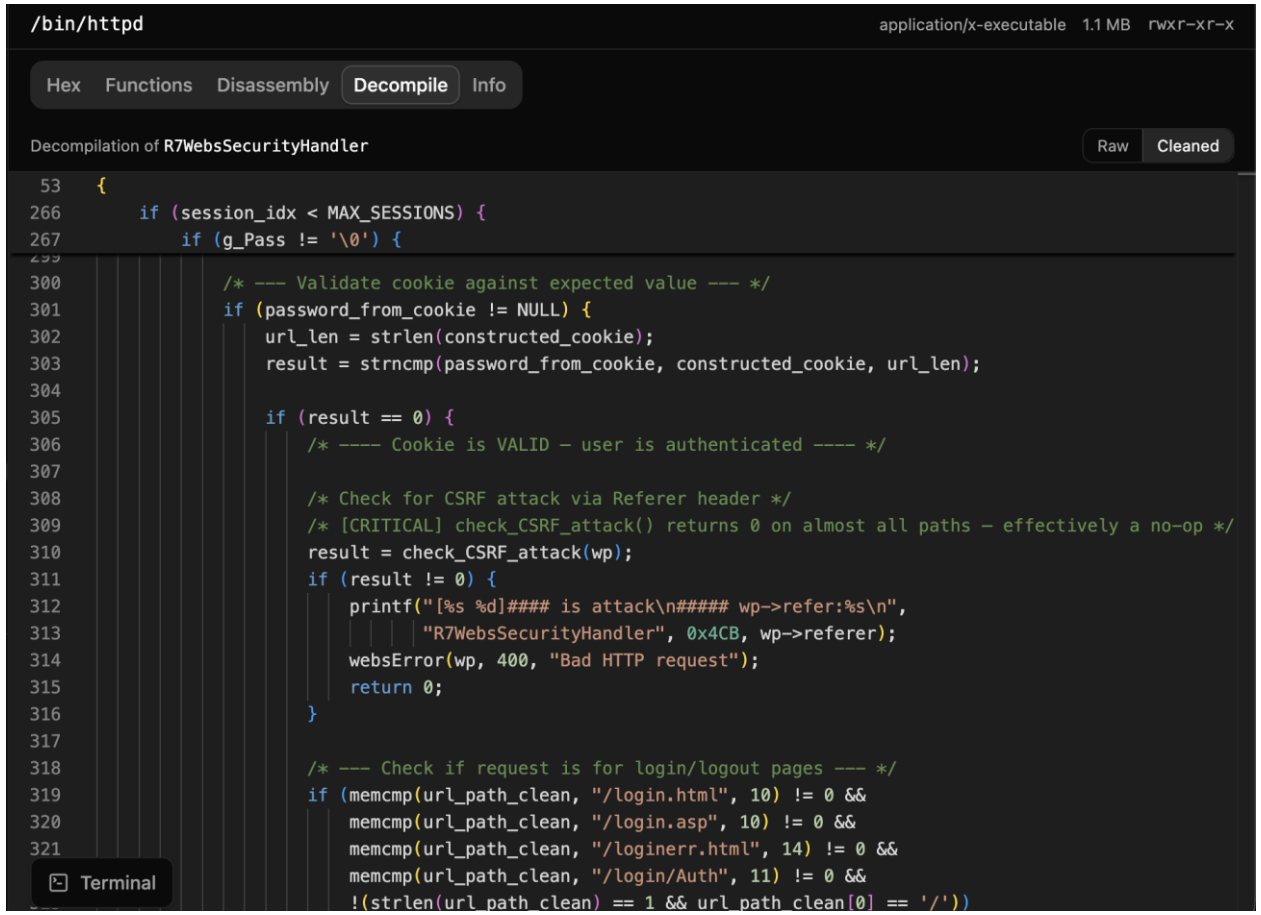
● Rich attack surface. Command-injection sinks (doSystemCmd) sit in many web handlers, plus the auth
  handler itself calls strcpy. Rather than burn context decompiling each huge function inline, I'll fan out
  three parallel analysis agents — each deep-dives a cluster via the wairz tools and reports back ranked
  findings.

  Running 3 agents… (ctrl+o to expand)
   ├ Audit auth/pre-auth surface in httpd · 9 tool uses · 32.2k tokens
   │ └ wairz - decompile_function (MCP)
   ├ Audit command-injection sinks in httpd · 3 tool uses · 19.3k tokens
   │ └ wairz - decompile_function (MCP)
   └ Audit stack overflow form handlers in httpd · 1 tool use · 12.8k tokens
     └ Initializing…
     (ctrl+b to run in background)
```

Anyone Can Hack IoT

wairz: binary analysis

- Ghidra headless + radare2 behind the agent
- decompile_function, list_imports, xrefs_to/from
- trace_dataflow & cross_binary_dataflow
- Agent can ”clean up” decompiled C
- **The agent reads the C with you**
- You can use straight decompiler with MCP as well





Anyone Can Hack IoT

wairz: emulation & fuzzing

EMULATION (QEMU)

- Boot firmware with no device on hand
- user-mode (single binary) or full system
- Isolated containers + GDB
- ARM / MIPS / MIPSel / AArch64
- Agent runs commands & reads logs

FUZZING (AFL++)

- QEMU mode — cross-arch binaries
- Auto dictionary + seed corpus
- Generates a fuzzing harness
- Crash triage built in
- Point it at the sink you just found



PART 4

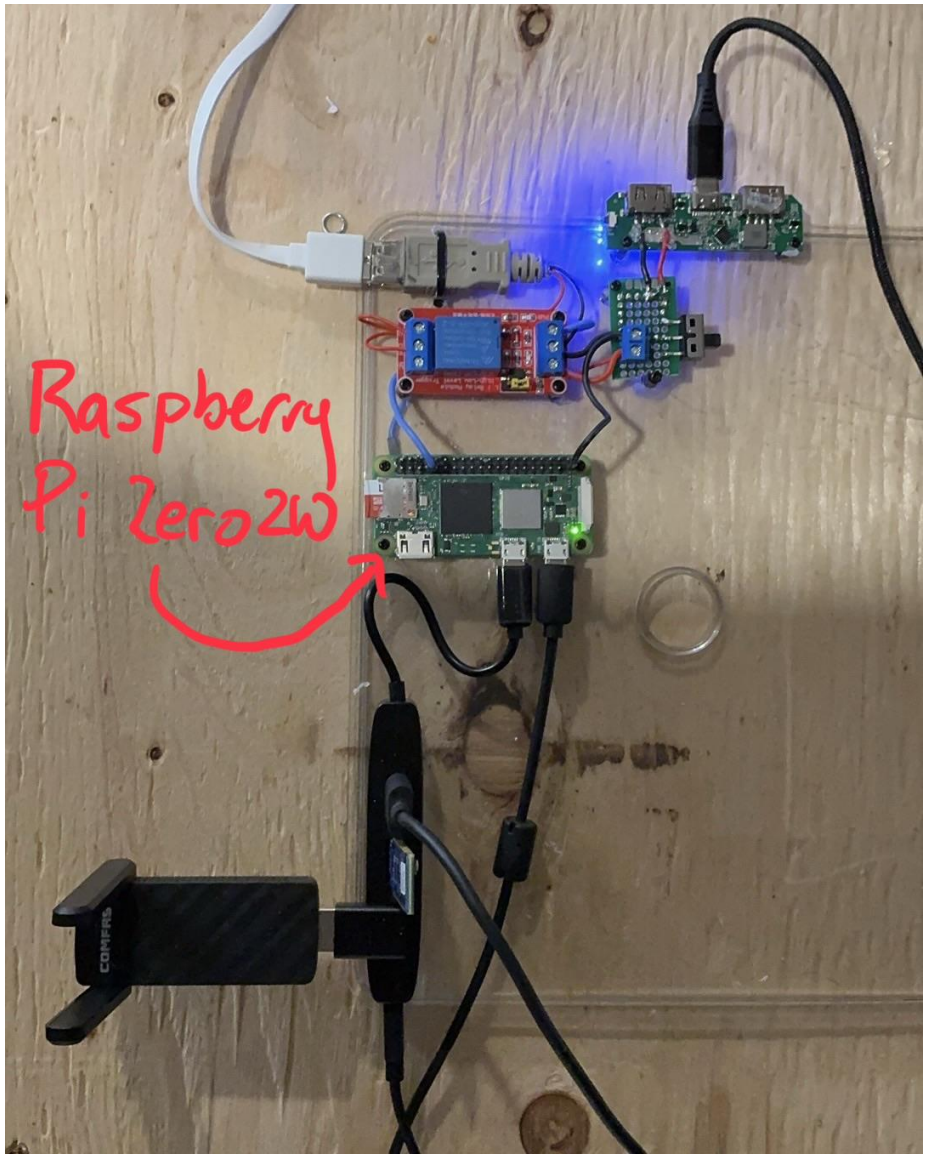
Bridging the Gap AI -> Hardware

I got tired of being asked to power it off and on again

Anyone Can Hack IoT

Giving AI direct access to hardware

- **I'm tired of power cycling for Claude…**
- Anything that blocks AI or scripting = how can I automate this
- First attempts = give coding agent access through my Linux desktop
- Didn't scale well, don't like AI having root, breaks things
- Started experimenting with Raspberry Pis
- AI gets ssh and sudo nopasswd
- Golden image of SD cards = quick fix
- **Goal: agent runs the repetitive bench loop, I supervise**

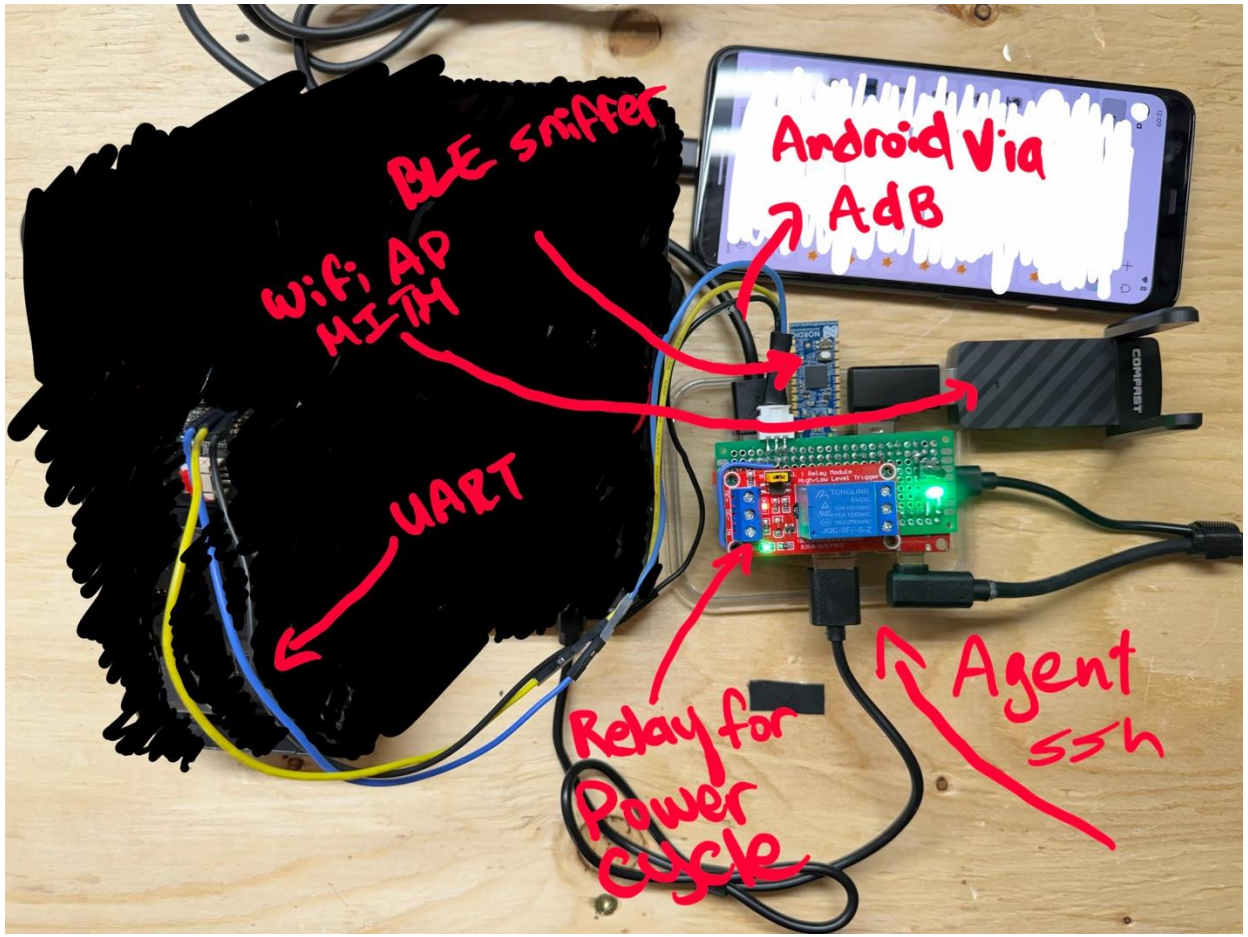


First Prototype

Anyone Can Hack IoT

Giving AI direct access to hardware

- **Start with the most impactful**
- Serial / UART consoles
- Relay to power cycle
- Alfa WiFi card (packet injection)
- Nordic NRF BLE Sniffer
- Android with ADB/Frida
- AI can RE and test live in a loop
- AI can write and test exploits/POCs
- **This covers 90% of my bench work**

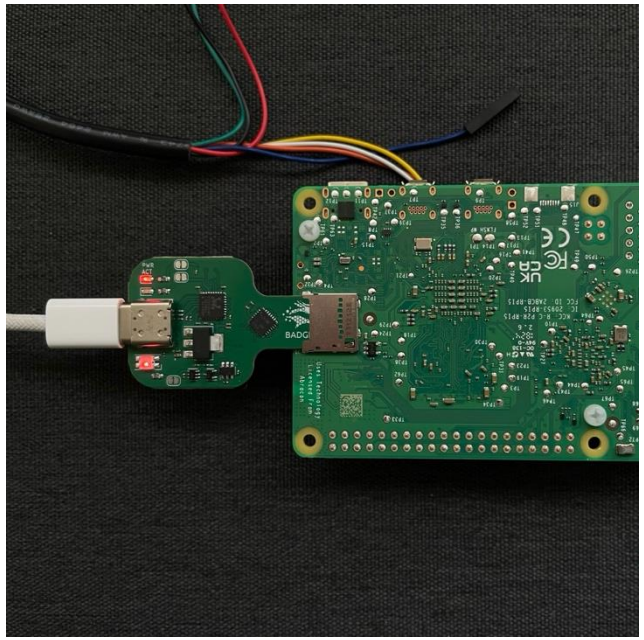


Anyone Can Hack IoT

Modularize the Extra 10%, here's some stuff I add in as needed



Tigard
SPI, JTAG, I2C



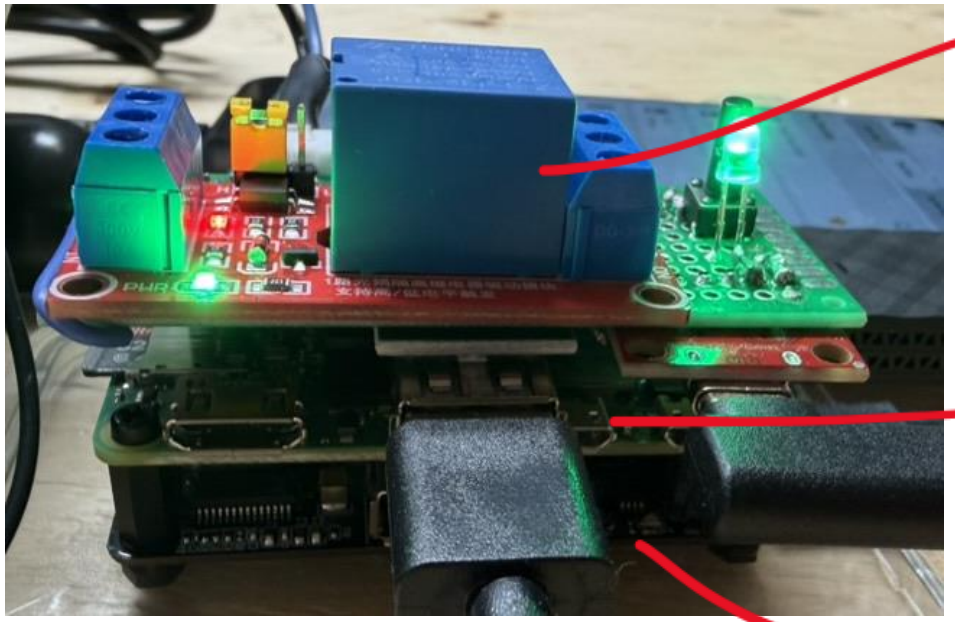
Badger SD-Wire

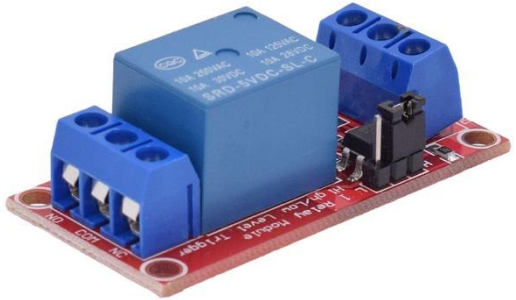


Cynthion

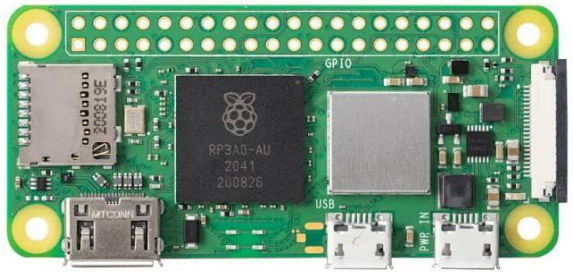
Anyone Can Hack IoT

Current state and parts to build yourself

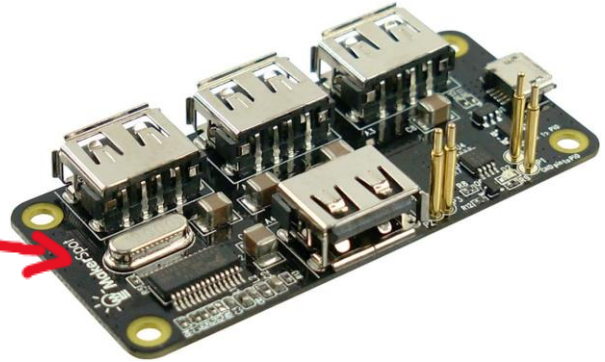




5V Relay
~$1



Pi Zero 2W
~$20-$40



Powered
USB Hat
~$10

Anyone Can Hack IoT

A real walkthrough: unbox → finding

1. $20 Tenda Router off Amazon (AI picked it)
   firmware dumped via SPI, UART password found by AI
2. wairz unpacks; agent flags custom httpd, beings RE
3. Agent traces web field → unsanitized system() and stack overflow
4. Agent writes POCs including ROP chain, tests on live device until it works, I verify at the end
5. Record finding, export report, disclose =
   CVE-2026-4254, CVE-2026-4253, CVE-2026-4252





PART 5

Resources and Wrap-up

Trust me, you can absolutely do this at home



Anyone Can Hack IoT

Cautions

Hallucinations

AI is still frequently confidently wrong, it could be an offset, a CVE, or a function. Verify every claim against real output.

Secrets & NDA

If you're doing this professionally, you gotta know where the data is going.

Skill atrophy

If you never learn to read the C yourself, you can't catch the agent when it's wrong.

Authorization

AI makes it fast to go too far. Same rule as always: only devices you're allowed to test.



Anyone Can Hack IoT

Resources & links

digitalandrew.io	Slides, blogs, links, all free
github.com/digitalandrew	Tools, scripts, resources, writeups
training.brownfinesecurity.com	My IoT hacking courses



Anyone Can Hack IoT

Key takeaways

- **Anyone can still hack IoT —> the bar is lower than ever**
- The methodology didn't change; AI automates the boring 80%
- Ground the AI in real tools, let it touch the hardware and iterate
- Automate reading & tedium; own judgment, hands, and trust
- **AI is cheap right now and open-source tools are all you need**
- **Go break something tonight**

FREE Hands-On Workshop - IoT Village This Afternoon



Thank you!

Now go buy a cheap device and hack it.

Andrew Bellini · @digitalandrew
digitalandrew.io · andrewbellini.com